
Mobile App Development

Lab Project Report

6th Semester

Course Code : CSE 618

Name: Tanjim Tajwar Arnab

Student ID: 22701066

Department: Computer Science and Engineering

University: University of Chittagong

Key Features

- Temperature Converter
- BMI Calculator
- General Calculator
- Islamic Inheritance
- Reverse String Tool
- To-Do List Manager
- Phone Call Simulator
- Zakah Calculator
- Email Sender Simulator
- Children Math Quiz

Submitted To

Prof. Dr. Mohammad Osiur Rahman
Professor, Computer Science and Engineering
University of Chittagong

April 14, 2026

Contents

Abstract	3
1 Setting Up the Development Environment	4
1.1 Install Flutter SDK	4
1.2 Install Visual Studio Code and Extensions	4
1.3 Create Flutter Project	5
2 Running and Testing the App	5
3 Understanding a Flutter Project Structure	6
4 Introduction	7
5 Technology Overview	9
5.1 Flutter Framework	9
5.2 Dart Programming Language	9
5.3 Development Tools (Android Studio / VS Code)	9
6 System Design	10
6.1 System Architecture	10
6.2 System Flow Diagram	11
7 Implementation of Features	11
7.1 Step-by-Step Workflow for Building One Feature	12
7.2 Temperature Converter	12
7.3 BMI Calculator	13
7.4 Calculator	13
7.5 Islamic Inheritance (Fara'id) App	14
7.6 Reverse String	14
7.7 To-Do List	14
7.8 Phone Call Simulator	14
7.9 Zakah Calculator	15
7.10 Email Sending App	15
7.11 Children Math App	15
8 Results and Discussion	16
8.1 App Performance	16
8.2 User Experience	16
8.3 Advantages of the System	16
9 Conclusion and Future Work	17
10 Testing and Validation	18
11 User Guide and Operational Manual	20
11.1 How to Use the Application	20
11.2 Feature-by-Feature Operating Instructions	20
11.3 Module Documentation Sheets	21
11.4 Maintenance and Update Guidelines	22

12 Repository and Version Control Documentation	24
12.1 GitHub Repository Overview	24
12.2 Future GitHub Workflow Plan	24
References	25
13 Application Screenshots	26
Appendix	27

List of Figures

1	System Flow Diagram of Amar Multiple App	11
2	Home Screen of the <i>Amar Multiple App</i> showing the main dashboard with all available modules.	26
3	BMI Calculator interface demonstrating user input fields and calculated health result display.	26
4	Temperature Converter module used for converting values between Celsius and Fahrenheit.	26
5	Children's Math Quiz module providing simple arithmetic questions for learning and practice.	27

List of Tables

1	Functional Test Cases for Core Features	18
---	---	----

Abstract

Amar Multiple App is a multi-functional mobile application developed using the Flutter framework, designed to integrate a variety of everyday utilities into a single unified platform. The application eliminates the need for multiple standalone apps by combining essential tools related to health, productivity, finance, and education within one user-friendly interface.

The system consists of ten core modules, including a Temperature Converter, BMI Calculator, General Calculator, Islamic Inheritance Calculator, Reverse String Tool, To-Do List Manager, Phone Call Simulator, Zakah Calculator, Email Sender Simulator, and a Children's Math Quiz. Each module is independently designed yet seamlessly integrated, demonstrating a modular and scalable architecture.

The application emphasizes fast performance through local processing, intuitive navigation using Flutter's routing system, and a clean, beginner-friendly user interface. This project highlights the practical implementation of software engineering principles, including modular design, state management, and formula-based logic, making it suitable for both academic learning and real-world application development.

1 Setting Up the Development Environment

How to Use This Report as a Tutorial

Read this report in order and implement each step while coding in VS Code. Do not skip setup validation commands. After every module, run the app and test output immediately. This practice helps beginners learn Flutter by doing, not only by reading.

Who This Tutorial Is For

This report is written for beginners who are new to Flutter, Dart, and mobile app development. If you can install software, run terminal commands, and edit files in VS Code, you can complete this project.

Learning Outcomes

By following this tutorial-style report, a beginner should be able to:

- Install and configure Flutter development tools.
- Create and run a Flutter app from scratch.
- Build multi-screen apps with route-based navigation.
- Handle user input, validation, and local state updates.
- Implement formula-based logic modules in Dart.
- Organize project code using reusable widgets and folders.

1.1 Install Flutter SDK

Download Flutter SDK from the official Flutter website and extract it to a local directory such as `C:\src\flutter`. Then add Flutter's `bin` path to environment variables and verify installation by running `flutter -version` and `flutter doctor`.

Quick Verification Steps

- Run `flutter -version` to confirm SDK availability.
- Run `flutter doctor` to detect missing dependencies.
- Complete all required setup suggestions shown by Flutter Doctor.

1.2 Install Visual Studio Code and Extensions

Install Visual Studio Code and then add the **Flutter** and **Dart** extensions from the Extensions Marketplace. These provide project templates, debugging support, code completion, and integrated Flutter commands.

Set Up an Android Emulator in VS Code (No Android Studio)

For a lightweight setup, configure emulator tooling from command-line Android SDK tools and control device launches from VS Code terminal.

Practical Setup Flow

1. Install Android SDK Command-line Tools.
2. Install platform tools (adb) and at least one system image.
3. Create an AVD from command line.
4. Start the emulator and verify with `flutter devices`.

1.3 Create Flutter Project

Create the project using:

```
1 flutter create amar_multiple_app
```

Listing 1: Create project command

After creation, enter the project directory and run `flutter pub get` if needed.

Run Application

Start the app from VS Code terminal using:

```
1 flutter run
```

Listing 2: Run app command

This launches the application on the selected emulator or connected device.

Beginner Checkpoint

Before moving to the next chapter, confirm these three items:

1. `flutter doctor` shows no critical errors.
2. Your emulator or phone appears in `flutter devices`.
3. The default Flutter counter app runs successfully.

2 Running and Testing the App

Run on Emulator

Open an Android emulator, ensure it appears in `flutter devices`, then execute `flutter run`. Use hot reload with `r` and hot restart with `R` in terminal for rapid testing cycles.

Deploy to Physical Android Phone via USB Tethering

Enable Developer Options and USB Debugging on the Android phone, connect via USB cable, approve RSA prompt, and confirm the device appears in `flutter devices`. Then run the app with `flutter run`.

Troubleshooting for Beginners

- If no device appears, reconnect cable and run `adb devices`.
- If build fails, run `flutter clean` then `flutter pub get`.
- If Gradle fails first time, wait for dependency download and rerun.
- If emulator is slow, close unused applications and cold boot emulator.

Testing Recommendation

Before final submission, validate all major modules (calculator, converter, inheritance, to-do, quiz, and communication simulators) on both emulator and physical phone to ensure stable behavior across environments.

3 Understanding a Flutter Project Structure

How to Read the Project Structure

Beginners should first understand `main.dart`, then `lib/screens/`, then `lib/widgets/`. This learning order makes Flutter architecture easier because you see app entry, navigation, feature screens, and shared UI components in sequence.

The `pubspec.yaml` File

This file is the central project manifest. It defines the project name, version, environment constraints, dependencies, dev dependencies, and asset configuration. Any new package integration or asset registration should be added here.

The `test` Folder

The `test` directory contains automated tests such as unit tests and widget tests. Even in small projects, basic tests improve reliability by validating expected behavior after code updates.

The `lib` Folder

The `lib` directory contains the main application source code. In this project, screens are organized under `lib/screens/` and reusable components are stored under `lib/widgets/`, supporting modular and maintainable development.

4 Introduction

Background of the Study

In recent years, mobile applications have become an essential part of everyday life. With the rapid growth of smartphones and internet accessibility, users rely heavily on mobile apps for performing daily tasks such as calculations, communication, education, and productivity management. As a result, there is a growing demand for applications that can integrate multiple functionalities into a single platform.

Flutter, developed by Google, has emerged as a powerful framework for building cross-platform mobile applications. It allows developers to create visually attractive and high-performance apps using a single codebase. This project utilizes Flutter to design and develop a multi-functional application named **Amar Multiple App**, which combines several useful tools and mini-applications into one unified system.

Unlike traditional applications that focus on a single functionality, this project integrates multiple utilities such as temperature conversion, BMI calculation, Islamic inheritance calculation, string manipulation, communication simulation, and educational tools. This approach enhances user convenience and reduces the need to install multiple separate applications.

Problem Statement

Most utility apps in app stores are designed around one task only. This creates several practical issues for users: increased storage consumption, repetitive user interfaces, inconsistent user experience, and frequent app switching. The **Amar Multiple App** addresses this by merging common utility functions into a single consistent environment.

Objective of the Project

The main objective of this project is to develop a comprehensive and user-friendly mobile application that integrates multiple useful features into a single platform. The specific objectives of the project are as follows:

- To design and develop a multi-functional mobile application using Flutter.
- To implement real-life utility features such as temperature conversion and BMI calculation.
- To incorporate Islamic financial tools such as inheritance and Zakah calculators.
- To create interactive and educational modules such as a children's math quiz.
- To simulate communication features like phone calls and email sending.
- To design an attractive, responsive, and user-friendly interface.
- To improve programming skills in Dart and Flutter framework.

Scope of the Project

The scope of this project is focused on developing a mobile application that combines multiple small-scale applications into one integrated system. The application is designed to be used by a wide range of users, including students, professionals, and general smartphone users.

The application includes ten major features:

- Temperature Converter (Celsius to Fahrenheit)

- BMI Calculator
- General Calculator
- Islamic Inheritance (Fara'id) Calculator
- Reverse String Tool
- To-Do List Manager
- Phone Call Simulation
- Zakah Calculator
- Email Sending Simulation
- Children Math Quiz

However, the project does not include advanced backend systems such as cloud storage, real-time communication, or authentication systems. All functionalities are implemented locally within the application.

Methodology

The development of this project follows a structured and systematic approach. The methodology used in this project includes the following steps:

1. **Requirement Analysis:** Identifying the features and functionalities to be included in the application.
2. **Design Phase:** Creating the user interface design and application structure.
3. **Development Phase:** Implementing the features using Flutter and Dart programming language.
4. **Testing Phase:** Testing each module to ensure proper functionality and user experience.
5. **Deployment:** Running the application on different platforms such as Android and Web.

This methodology ensures that the application is developed efficiently and meets the intended objectives.

Software Development Lifecycle Used

1. **Planning:** Defined module list, user flow, and minimum functional requirements.
2. **UI Prototyping:** Designed card-based home layout and reusable form patterns.
3. **Iterative Coding:** Built each module independently and validated output logic.
4. **Integration:** Connected all modules through centralized navigation from home screen.
5. **Validation:** Performed manual test cases for normal, boundary, and invalid inputs.

5 Technology Overview

5.1 Flutter Framework

Flutter is an open-source UI software development kit (SDK) developed by Google. It is widely used for building cross-platform applications for mobile, web, and desktop using a single codebase. Flutter uses the Dart programming language and provides a rich set of pre-designed widgets that help developers create visually appealing user interfaces.

One of the key advantages of Flutter is its **hot reload** feature, which allows developers to see changes in real-time without restarting the application. This significantly improves development speed and productivity.

Flutter follows a widget-based architecture, where everything in the application is considered a widget, including layouts, buttons, text, and even the entire screen. This approach makes the UI highly customizable and flexible.

In this project, Flutter is used to design and implement all the user interfaces and navigation systems. The framework enables smooth animations, responsive layouts, and consistent design across different devices.

5.2 Dart Programming Language

Dart is a modern, object-oriented programming language developed by Google. It is the primary language used in Flutter development. Dart is known for its simplicity, efficiency, and strong typing system.

Some important features of Dart include:

- **Object-Oriented Programming:** Supports classes, inheritance, and encapsulation.
- **Asynchronous Programming:** Provides support for `async` and `await` for handling tasks such as delays and API calls.
- **Strong Typing:** Helps in reducing runtime errors.
- **Fast Execution:** Compiles to native code for better performance.

In this project, Dart is used to implement the application logic, including calculations, data processing, and user interactions. It ensures efficient and smooth execution of all features.

5.3 Development Tools (Android Studio / VS Code)

The development of this project is carried out using modern and efficient development tools. The primary tools used are:

Visual Studio Code (VS Code): VS Code is a lightweight and powerful code editor that provides excellent support for Flutter and Dart development. It includes features such as syntax highlighting, debugging tools, extensions, and integrated terminal support.

Android Studio: Android Studio is an official Integrated Development Environment (IDE) for Android development. It provides advanced tools for building, testing, and debugging mobile applications. It also includes an emulator for testing applications without a physical device.

Flutter SDK: The Flutter Software Development Kit (SDK) is used to build and run the application. It includes libraries, tools, and frameworks required for Flutter development.

Browser (Chrome/Edge): The application is also tested on web platforms using modern browsers like Google Chrome and Microsoft Edge.

These tools collectively help in efficient development, testing, and deployment of the application.

Why This Technology Stack

The selected stack is optimized for rapid prototyping and clean user interface development. Flutter provides reusable widgets and high rendering performance, Dart keeps logic readable and maintainable, and VS Code/Android Studio support fast debugging and testing. This combination is ideal for student projects that need both development speed and practical quality.

6 System Design

6.1 System Architecture

The system architecture of the **Amar Multiple App** is designed using a modular approach. Each feature of the application is developed as an independent module, allowing better organization, maintainability, and scalability.

The application follows a simple layered architecture:

- **Presentation Layer:** This layer is responsible for the user interface. It includes all screens such as Home Screen, BMI Screen, Temperature Screen, and others.
- **Logic Layer:** This layer handles all the calculations and processing logic, such as BMI computation, temperature conversion, and inheritance calculation.
- **Navigation Layer:** This layer manages the routing between different screens using Flutter's navigation system.

Each module interacts with the user independently, ensuring that the failure of one module does not affect the overall application.

User Interface Design

The user interface of the application is designed to be modern, colorful, and user-friendly. Special attention has been given to ensure a consistent design across all screens.

The following design principles are followed:

- **Consistency:** All screens use a similar color scheme, typography, and layout structure.
- **Responsiveness:** The application adapts to different screen sizes and devices.
- **Simplicity:** The interface is kept simple so that users can easily understand and use the features.
- **Visual Appeal:** Gradients, icons, and cards are used to enhance the overall look of the application.

The home screen acts as a dashboard that provides access to all features through a grid layout. Each feature is represented with an icon and color combination to make it easily identifiable.

Navigation Flow

Navigation in the application is handled using Flutter's routing system. Each feature is assigned a unique route name, which allows easy navigation between screens.

The navigation flow works as follows:

1. The user starts at the Home Screen.
2. The user selects a feature from the grid.
3. The application navigates to the selected feature screen.
4. After completing the task, the user can return to the Home Screen.

This simple navigation structure ensures a smooth and intuitive user experience.

Design Decisions and Rationale

- **Dashboard Entry Point:** Reduces interaction cost by showing all modules on first screen.
- **Module Independence:** Minimizes coupling, making updates easier and safer.
- **Form-Centric Screens:** Most tools rely on input/output forms, so a consistent pattern improves usability.
- **Local Computation:** Keeps app fast and functional without internet dependency.

6.2 System Flow Diagram

The overall workflow of the application can be described as follows:

- User opens the application.
- Home Screen displays all available features.
- User selects a feature.
- Input is provided by the user.
- The system processes the input.
- Output/result is displayed.

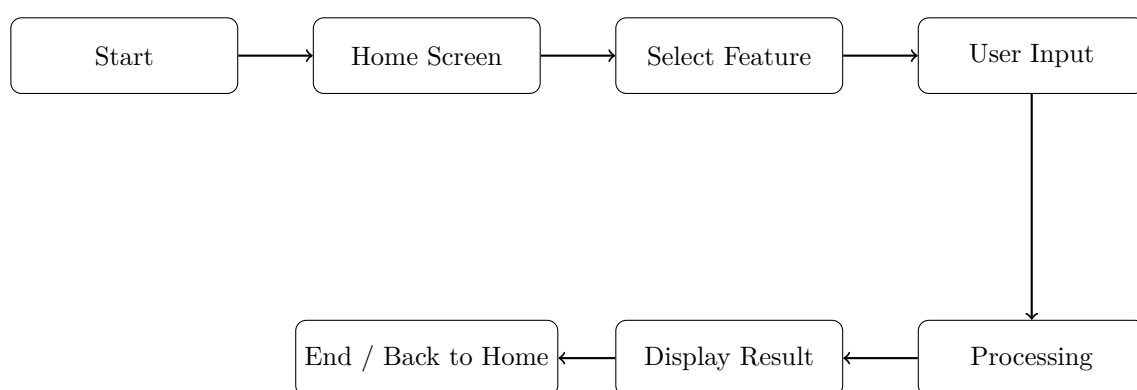


Figure 1: System Flow Diagram of Amar Multiple App

7 Implementation of Features

This chapter describes the implementation details of each feature included in the **Amar Multiple App**. Each module is designed as an independent unit and performs a specific task.

Build Pattern Used in Every Module

To help beginners, each feature follows the same development pattern:

1. Create a new screen class in `lib/screens/`.
2. Add input fields and action button.
3. Write logic method (convert/calculate/reverse/check).
4. Validate input and show feedback for invalid cases.
5. Display result using a styled result card.
6. Register route in `main.dart` and add home-screen card.

7.1 Step-by-Step Workflow for Building One Feature

Use the following checklist each time you create a new feature screen:

1. Create a Dart file in `lib/screens/feature_name_screen.dart`.
2. Add `static const routeName`.
3. Build UI with `Scaffold`, input fields, button, and result area.
4. Write one method to process user input.
5. Add `TextEditingController` and dispose it properly.
6. Register the screen route in `main.dart`.
7. Add the feature card in `home_screen.dart`.
8. Run app and test normal, boundary, and invalid input cases.

7.2 Temperature Converter

The Temperature Converter module allows users to convert temperature from Celsius to Fahrenheit.

The user inputs a temperature value in Celsius, and the system calculates the equivalent Fahrenheit value using a mathematical formula.

Key Features:

- Simple input field for Celsius value
- Instant calculation
- Clean and readable output display

Working Principle: The conversion is performed using the formula:

$$F = (C \times \frac{9}{5}) + 32$$

User Interface: The interface consists of an input field, a button, and a result display container.

Implementation Details

Input: Celsius value from user text field.

Processing: Parse value as double and apply conversion formula.

Validation: Empty or invalid input is prevented with message feedback.

Output: Formatted Fahrenheit value shown instantly in result area.

Try It Yourself

Extend this module to support Fahrenheit-to-Celsius conversion. Add a toggle button for conversion direction and update formula logic accordingly.

7.3 BMI Calculator

The BMI (Body Mass Index) Calculator is used to determine whether a person is underweight, normal, overweight, or obese.

Key Features:

- Input fields for height and weight
- Automatic BMI calculation
- Category classification with color indication

Working Principle: BMI is calculated using:

$$BMI = \frac{Weight}{Height^2}$$

User Interface: The UI provides fields for height and weight along with a result section that displays BMI value and category.

Implementation Details

Input: Height and weight values.

Processing: Converts units and calculates BMI.

Validation: Non-positive values are rejected.

Output: Numeric BMI with category label (Underweight/Normal/Overweight/Obese).

Try It Yourself

Add support for imperial units (feet/inches and pounds) and allow users to switch between metric and imperial modes.

7.4 Calculator

The Calculator module performs basic arithmetic operations such as addition, subtraction, multiplication, and division.

Key Features:

- User-friendly keypad
- Supports multiple operations

- Instant result calculation

User Interface: The interface includes numeric buttons, operator buttons, and a display screen for showing results.

7.5 Islamic Inheritance (Fara'id) App

This module calculates the distribution of wealth among heirs based on Islamic inheritance rules.

Key Features:

- Input for number of sons and daughters
- Automatic share calculation
- Clear result breakdown

Working Principle: Each son receives double the share of each daughter. The total wealth is divided based on this ratio.

User Interface: The interface includes input fields and a result section showing the calculated shares.

7.6 Reverse String

This module allows users to reverse any input text.

Key Features:

- Simple text input
- Instant reversal
- Clean output display

Working Principle: The input string is reversed using string manipulation techniques.

7.7 To-Do List

The To-Do List module helps users manage daily tasks.

Key Features:

- Add tasks
- Delete tasks
- Mark tasks as completed

User Interface: The UI displays a list of tasks with options to add and remove items.

7.8 Phone Call Simulator

This module simulates a phone call interface.

Key Features:

- Input phone number
- Simulated incoming call screen
- Accept and decline options

Working Principle: The module does not perform real calls but simulates a call interface for demonstration purposes.

7.9 Zakah Calculator

The Zakah Calculator helps users calculate the amount of Zakah based on their wealth.

Key Features:

- Input total wealth
- Automatic calculation
- Simple and clear output

Working Principle: Zakah is calculated as 2.5% of total wealth.

7.10 Email Sending App

This module simulates sending an email.

Key Features:

- Input email, subject, and message
- Simulated email notification
- Simple interface

Working Principle: The app opens the email client and displays a simulated message indicating that an email has been received.

7.11 Children Math App

This module is designed for children to practice basic math.

Key Features:

- Random math questions
- Score tracking
- Instant feedback

Working Principle: Random numbers are generated, and users solve simple addition problems.

8 Results and Discussion

8.1 App Performance

The developed application, **Amar Multiple App**, performs efficiently across different platforms, including mobile and web environments. All features respond quickly to user inputs, and calculations are performed instantly without noticeable delay.

The use of Flutter ensures smooth rendering of user interfaces and consistent performance across devices. The application does not require heavy system resources, making it suitable for a wide range of devices.

Observed Performance Indicators

- Fast screen transitions due to lightweight routing.
- Near-instant formula-based calculations across utility modules.
- Stable behavior in repeated input/output cycles.
- Minimal memory load because of local-only data processing.

8.2 User Experience

User experience is one of the key strengths of this application. The interface is designed to be simple, colorful, and easy to navigate. Each feature is clearly labeled, allowing users to quickly understand and access different functionalities.

The use of cards, gradients, icons, and responsive layouts enhances the visual appeal of the application. Feedback mechanisms such as messages, result displays, and interactive buttons improve user interaction.

Additionally, the home screen acts as a centralized dashboard, enabling users to access all features from a single place, which significantly improves usability.

8.3 Advantages of the System

The developed system offers several advantages:

- **Multi-functionality:** Combines multiple useful applications into one platform.
- **User-Friendly Interface:** Simple and intuitive design suitable for all users.
- **Lightweight Application:** Does not require high system resources.
- **Cross-Platform Support:** Runs on Android and web platforms.
- **Educational Value:** Includes learning tools such as the math quiz.
- **Practical Utility:** Provides real-life tools like BMI, Zakah, and inheritance calculators.

Limitations

The current version prioritizes functional clarity over production-grade architecture. Persistent storage, automated testing pipelines, and backend integration are intentionally excluded to keep the first release focused and manageable.

9 Conclusion and Future Work

Conclusion

In conclusion, the **Amar Multiple App** successfully demonstrates the development of a multi-functional mobile application using Flutter. The application integrates various tools and utilities into a single platform, providing convenience and efficiency for users.

The project highlights the power of Flutter in creating attractive and responsive user interfaces. It also demonstrates the effective use of Dart programming for implementing logic and functionality.

Through this project, important concepts such as UI design, state management, user interaction, and modular development have been explored and implemented. The application meets its objectives and serves as a practical solution for combining multiple features in one system.

Future Improvements

Although the application is functional and complete, there are several areas for future improvement:

- **User Authentication:** Adding login and registration features.
- **Cloud Integration:** Storing data using Firebase or other cloud services.
- **Advanced UI Design:** Adding animations and themes (dark/light mode).
- **Real Communication Features:** Implementing real phone call and email functionalities.
- **Database Support:** Saving user data such as tasks and quiz scores.
- **Multilingual Support:** Supporting multiple languages for wider accessibility.

These improvements can enhance the functionality and usability of the application in future versions.

Recommended Development Roadmap

Phase 1: Local persistence with SQLite/SharedPreferences for tasks and scores.

Phase 2: User account and cloud synchronization.

Phase 3: Advanced UI polish, theming, and accessibility support.

Phase 4: Full testing strategy (unit, widget, and integration tests) with CI.

10 Testing and Validation

Testing Strategy

To ensure reliability and correctness, testing was performed in a module-by-module manner, followed by integrated application testing. The strategy emphasized three types of validation: functional correctness, usability, and error handling. Each feature was tested with normal, boundary, and invalid inputs to verify that outputs remained stable and meaningful.

Validation Objectives

- Verify that each formula and logic path returns expected output.
- Confirm that user interface controls respond correctly.
- Ensure invalid input does not crash the application.
- Check whether navigation remains smooth between all modules.
- Evaluate clarity of messages, labels, and result presentation.

Functional Test Cases

The following test matrix summarizes selected functional test cases performed for key modules.

Table 1: Functional Test Cases for Core Features

Module		Input	Expected Output	Status
Temperature	Con-	0 Celsius	32 Fahrenheit	Pass
verter				
Temperature	Con-	100 Celsius	212 Fahrenheit	Pass
verter				
BMI Calculator		70 kg, 1.75 m	BMI $\approx$ 22.86 (Normal)	Pass
BMI Calculator		0 kg, 1.75 m	Validation error message	Pass
Calculator		12 + 8	20	Pass
Calculator		12 / 0	Division error handling	Pass
Inheritance		Wealth 90000, 1 son, 1 daughter	Son gets double daughter share	Pass
Reverse String		“Flutter”	“rettulF”	Pass
To-Do List		Add a task	Task appears in list	Pass
Zakah Calculator		Wealth 100000	Zakah 2500	Pass

Input Validation and Error Handling

Input validation is a critical quality requirement for utility applications. Since most modules depend on user-provided numeric values, validation prevents incorrect parsing, misleading output, and runtime exceptions. In this project, validation logic was implemented before calculations and navigation actions.

Error Handling Principles Followed

- Empty fields trigger user-friendly prompts.
- Non-numeric values are rejected before formula evaluation.
- Negative values are restricted where logically invalid.

- Division-by-zero and equivalent undefined states are safely handled.
- Feedback messages are kept concise and understandable.

Usability Evaluation

Usability was assessed through repeated interactions with all modules from the perspective of first-time users. The key criteria included time to complete a task, number of taps required, readability of outputs, and ease of returning to the home dashboard.

The card-based home interface helped users discover all features quickly. In most modules, tasks could be completed in three simple stages: input, process, and result. This predictable structure significantly reduced confusion and improved confidence while using the app.

Usability Observations

Strengths:

- Consistent screen layout across modules.
- Immediate result generation for formula-based tools.
- Clear separation between input controls and output region.
- Friendly visual style with high readability.

Areas to Improve:

- Add helper text for technical terms (e.g., BMI categories).
- Improve accessibility with larger text options.
- Add confirmation dialogs before destructive actions.

Cross-Platform Validation

The application was tested in Android runtime and web runtime to observe rendering consistency, control behavior, and interaction latency. Flutter's unified rendering model maintained consistent visual appearance, while logical outputs remained identical across both platforms.

Cross-Platform Result Summary

- Core logic produced same outputs across platforms.
- Layout remained stable in common screen sizes.
- Input controls and buttons behaved consistently.
- Navigation flow remained identical in Android and web testing.

11 User Guide and Operational Manual

Tutorial Completion Goal

After finishing this chapter, a beginner should be able to install Flutter tools, create a multi-screen app, implement input-processing features, test on emulator and phone, and document the project in report form.

11.1 How to Use the Application

This section provides step-by-step instructions to help users operate each module of the **Amar Multiple App**. The objective is to make the app accessible to users with minimal technical background.

General Usage Flow

1. Open the application and wait for the home dashboard.
2. Select a feature card based on your task.
3. Enter required input values.
4. Press the action button (Calculate, Convert, Send, or Check).
5. Read the displayed output or feedback.
6. Use back navigation to return to the home screen.

11.2 Feature-by-Feature Operating Instructions

Temperature Converter: Enter temperature in Celsius and tap convert. The app calculates Fahrenheit instantly and displays it below the button.

BMI Calculator: Enter weight and height values. Tap calculate to view BMI score and health classification.

Calculator: Use keypad buttons to build an expression. Press equals to evaluate and clear to reset.

Islamic Inheritance: Provide total wealth and number of heirs. The app calculates shares according to configured ratio logic.

Reverse String: Type text and tap reverse. The reversed output appears immediately.

To-Do List: Add task text, press add, and manage status by marking completed or deleting.

Phone Simulator: Enter phone number and simulate call behavior through accept or decline controls.

Zakah Calculator: Enter total wealth and press calculate to get Zakah amount at 2.5 percent.

Email Sender: Enter recipient, subject, and message body. Trigger send action to open prefilled draft behavior.

Children Math Quiz: Solve generated question, submit answer, and observe score updates.

11.3 Module Documentation Sheets

Temperature Converter Documentation

Purpose: Convert Celsius values to Fahrenheit for daily usage.

Input Type: Decimal numeric.

Output Type: Decimal numeric with unit label.

Common Error: Empty input.

Suggested Improvement: Add bi-directional conversion (Fahrenheit to Celsius).

BMI Calculator Documentation

Purpose: Compute Body Mass Index and classify category.

Input Type: Weight and height numeric values.

Output Type: BMI value + category.

Common Error: Wrong unit assumption.

Suggested Improvement: Add metric/imperial unit switch.

Calculator Documentation

Purpose: Perform everyday arithmetic.

Input Type: Digits and operators.

Output Type: Expression result.

Common Error: Incomplete expression.

Suggested Improvement: Add scientific operations.

Inheritance Documentation

Purpose: Provide educational inheritance share estimation.

Input Type: Wealth and heir count.

Output Type: Share distribution.

Common Error: Unrealistic heir input.

Suggested Improvement: Add complete jurisprudence scenarios.

Reverse String Documentation

Purpose: Reverse arbitrary text input.

Input Type: Plain text.

Output Type: Reversed plain text.

Common Error: Empty string submission.

Suggested Improvement: Add case-transform tools.

To-Do List Documentation

Purpose: Track daily tasks in memory.

Input Type: Task text.

Output Type: Interactive checklist.

Common Error: Duplicate task entries.

Suggested Improvement: Add persistent local storage.

Phone Simulator Documentation

Purpose: Demonstrate call UI interactions.

Input Type: Phone number string.

Output Type: Simulated call state.

Common Error: Non-digit symbols.

Suggested Improvement: Add fake call history logs.

Zakah Calculator Documentation

Purpose: Compute payable Zakah.

Input Type: Wealth numeric value.

Output Type: Zakah amount.

Common Error: Misunderstanding nisab threshold.

Suggested Improvement: Add nisab and asset-type breakdown.

Email Sender Documentation

Purpose: Demonstrate email composition flow.

Input Type: Recipient, subject, message.

Output Type: Prefilled email draft action.

Common Error: Invalid email format.

Suggested Improvement: Add attachment support.

Children Math Quiz Documentation

Purpose: Help children practice arithmetic.

Input Type: Numeric answer.

Output Type: Correct/incorrect feedback + score.

Common Error: Non-numeric answer text.

Suggested Improvement: Add levels, timers, and progress analytics.

11.4 Maintenance and Update Guidelines

To keep the project maintainable over time, each module should be updated independently with minimal side effects on other modules. Any logic update should be followed by quick regression testing of input validation and output formatting.

Recommended Maintenance Checklist

- Keep each feature in separate screen and logic units.
- Use reusable widget patterns to avoid duplicated UI code.
- Standardize validation messages for consistency.
- Update documentation whenever formulas or workflows change.
- Add unit tests when introducing new business logic.

Project Reflection and Learning Outcomes

Technical Learning Outcomes

This project provided practical experience in designing and implementing a medium-scale Flutter application using a modular architecture. The development process strengthened understanding of widget composition, navigation routing, user input processing, and local state updates.

From an engineering perspective, the most significant learning was balancing design quality with implementation simplicity. Several features required careful validation logic to avoid runtime instability. Through these scenarios, the project reinforced the importance of defensive programming and user-centered interface behavior.

Academic and Practical Value

Academically, this project demonstrates applied software engineering in a realistic context by connecting design, implementation, and evaluation phases into one complete product lifecycle. Practically, it offers direct utility for daily tasks and can be expanded into a more advanced system in future iterations.

Future Research Direction

- Integration of personalization and recommendation logic.
- Comparative UX study between single-purpose and multi-purpose apps.
- Performance profiling for low-end devices.
- Cloud-sync architecture for task and score portability.

12 Repository and Version Control Documentation

12.1 GitHub Repository Overview

Official Repository Link

<https://github.com/TanjimTajwar/myFlutterWorks.git>

This repository helps document project evolution and allows the report evaluator to verify that the implementation discussed in this report corresponds to real development artifacts.

Why Version Control Was Important

Using Git and GitHub improved development discipline in several ways:

- It preserved incremental progress through structured commits.
- It reduced risk of losing code during experiments and refactoring.
- It enabled easier rollback when a change introduced unexpected behavior.
- It created an auditable timeline of design and implementation decisions.
- It provides a foundation for collaboration in future team-based work.

Repository Structure Perspective

At a high level, the repository contains multiple Flutter learning and project folders, including this application. For the **Amar Multiple App**, the most relevant directories are:

- `lib/`: Core application source code.
- `lib/screens/`: Feature-specific screens and interaction logic.
- `lib/widgets/`: Reusable UI components.
- `assets/`: Static resources such as icons or images (if included).

Traceability Benefit

By combining report documentation with a public repository reference, this project becomes easier to evaluate for authenticity, reproducibility, and maintainability. Any reviewer can cross-check architectural descriptions, feature implementation, and coding style directly from the source.

12.2 Future GitHub Workflow Plan

For future versions, repository practices can be further improved through:

1. Semantic commit messages for clearer change intent.
2. Separate development and release branches.
3. Pull request-based review before merging major updates.
4. Issue tracking for bugs and enhancement planning.
5. Continuous Integration (CI) for automated testing.

References

1. Google Developers, *Flutter Documentation*. Available at: <https://docs.flutter.dev/>
2. Dart Programming Language, *Official Documentation*. Available at: <https://dart.dev/guides>
3. Android Developers, *Android Studio User Guide*. Available at: <https://developer.android.com/studio>
4. Visual Studio Code, *Official Documentation*. Available at: <https://code.visualstudio.com/docs>
5. BMI Calculation, World Health Organization (WHO). Available at: <https://www.who.int>
6. Islamic Inheritance Laws (Fara'id), Islamic Studies Resources. Available at: <https://www.islamicfinder.org>
7. Zakah Calculation Guidelines. Available at: <https://www.zakat.org>
8. Flutter Packages, *URL Launcher Package*. Available at: https://pub.dev/packages/url_launcher
9. Software Engineering Concepts, *System Design Basics*. Available at: <https://www.geeksforgeeks.org>
10. Mobile Application Development Concepts, *General Overview*. Available at: <https://www.tutorialspoint.com>
11. Tanjim Tajwar, *myFlutterWorks Repository*. Available at: <https://github.com/TanjimTajwar/myFlutterWorks.git>

13 Application Screenshots

This section presents the user interface of different modules of the application.

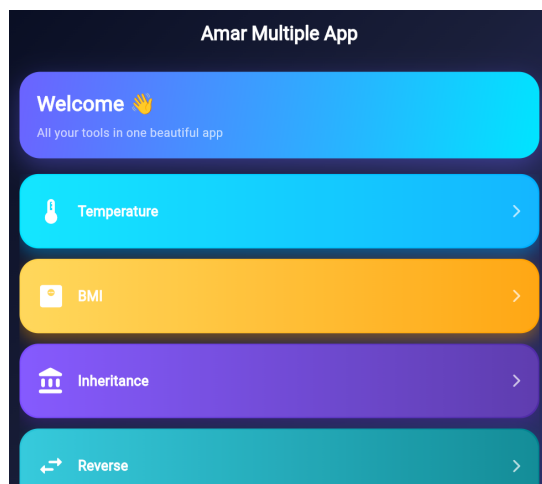


Figure 2: Home Screen of the *Amar Multiple App* showing the main dashboard with all available modules.

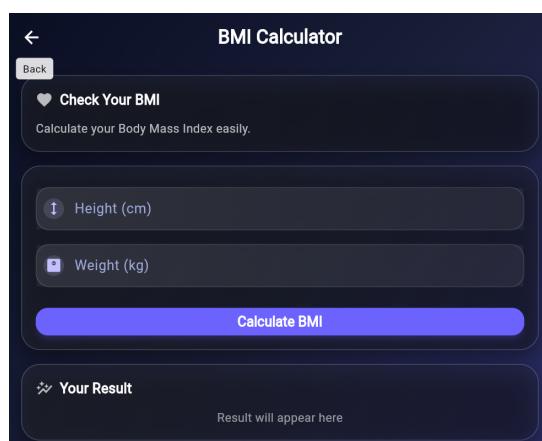


Figure 3: BMI Calculator interface demonstrating user input fields and calculated health result display.

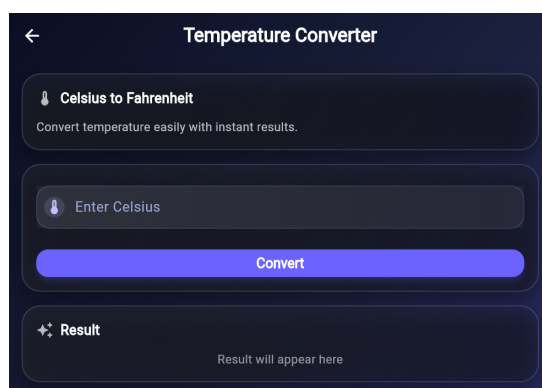


Figure 4: Temperature Converter module used for converting values between Celsius and Fahrenheit.

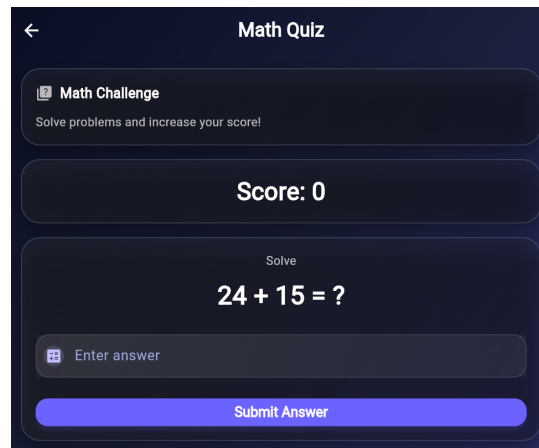


Figure 5: Children’s Math Quiz module providing simple arithmetic questions for learning and practice.

Appendix

Project Source Code from lib/

This appendix includes actual source files used in the **Amar Multiple App** project. The listings are inserted directly from the project directory so the report reflects the real implementation.

Code Inclusion Note

The following code blocks are presented directly using `\begin{lstlisting}` so the appendix remains visible even when external file loading is unavailable.

Main Application Entry

```

1  import 'package:flutter/material.dart';
2  import 'screens/bmi_screen.dart';
3  import 'screens/calculator_screen.dart';
4  import 'screens/call_screen.dart';
5  import 'screens/email_screen.dart';
6  import 'screens/home_screen.dart';
7  import 'screens/inheritance_screen.dart';
8  import 'screens/quiz_screen.dart';
9  import 'screens/reverse_screen.dart';
10 import 'screens/temperature_screen.dart';
11 import 'screens/todo_screen.dart';
12 import 'screens/zakah_screen.dart';
13
14 void main() {
15   runApp(const AmarMultipleApp());
16 }
17 class AmarMultipleApp extends StatelessWidget {
18   const AmarMultipleApp({super.key});
19   @override
20   Widget build(BuildContext context) {

```

```

21   final colorScheme = ColorScheme.fromSeed(
22     seedColor: const Color(0xFF6C63FF), // modern purple-blue
23     brightness: Brightness.dark,
24   );
25   return MaterialApp(
26     debugShowCheckedModeBanner: false,
27     title: 'Amar Multiple App',
28     theme: ThemeData(
29       useMaterial3: true,
30       colorScheme: colorScheme,
31       scaffoldBackgroundColor: const Color(0xFF0A0F24),
32     ),
33     routes: {
34       HomeScreen.routeName: (_) => const HomeScreen(),
35       TemperatureScreen.routeName: (_) => const TemperatureScreen(),
36       BmiScreen.routeName: (_) => const BmiScreen(),
37       InheritanceScreen.routeName: (_) => const InheritanceScreen(),
38       ReverseScreen.routeName: (_) => const ReverseScreen(),
39       CallScreen.routeName: (_) => const CallScreen(),
40       EmailScreen.routeName: (_) => const EmailScreen(),
41       QuizScreen.routeName: (_) => const QuizScreen(),
42       TodoScreen.routeName: (_) => const TodoScreen(),
43       CalculatorScreen.routeName: (_) => const CalculatorScreen(),
44       ZakahScreen.routeName: (_) => const ZakahScreen(),
45     },
46     initialRoute: HomeScreen.routeName,
47   );
48 }
49 }

```

Listing 3: lib/main.dart

Screens

```

1  import 'package:flutter/material.dart';
2
3  import 'bmi_screen.dart';
4  import 'calculator_screen.dart';
5  import 'call_screen.dart';
6  import 'email_screen.dart';
7  import 'inheritance_screen.dart';
8  import 'quiz_screen.dart';
9  import 'reverse_screen.dart';
10 import 'temperature_screen.dart';
11 import 'todo_screen.dart';
12 import 'zakah_screen.dart';
13
14 class HomeScreen extends StatelessWidget {
15   const HomeScreen({super.key});

```

```
16
17   static const routeName = '/';
18
19   @override
20   Widget build(BuildContext context) {
21     return const Scaffold(
22       body: Center(child: Text('Home Screen')),
23     );
24   }
25 }
```

Listing 4: lib/screens/home_screen.dart

```
1  import 'package:flutter/material.dart';
2  import '../widgets/custom_card.dart';
3  import '../widgets/custom_textfield.dart';
4  class TemperatureScreen extends StatefulWidget {
5    const TemperatureScreen({super.key});
6    static const routeName = '/temperature';
7    @override
8    State<TemperatureScreen> createState() => _TemperatureScreenState();
9  }
```

Listing 5: lib/screens/temperature_screen.dart

```
1  import 'package:flutter/material.dart';
2  import '../widgets/custom_card.dart';
3  import '../widgets/custom_textfield.dart';
4  class BmiScreen extends StatefulWidget {
5    const BmiScreen({super.key});
6    static const routeName = '/bmi';
7    @override
8    State<BmiScreen> createState() => _BmiScreenState();
9  }
```

Listing 6: lib/screens/bmi_screen.dart

```
1  import 'package:flutter/material.dart';
2  class CalculatorScreen extends StatefulWidget {
3    const CalculatorScreen({super.key});
4    static const routeName = '/calculator';
5    @override
6    State<CalculatorScreen> createState() => _CalculatorScreenState();
7  }
```

Listing 7: lib/screens/calculator_screen.dart

```
1 import 'package:flutter/material.dart';
2 import '../widgets/custom_card.dart';
3 import '../widgets/custom_textfield.dart';
4 class InheritanceScreen extends StatefulWidget {
5   const InheritanceScreen({super.key});
6   static const routeName = '/inheritance';
7   @override
8   State<InheritanceScreen> createState() => _InheritanceScreenState();
9 }
```

Listing 8: lib/screens/inheritance_screen.dart

```
1 import 'package:flutter/material.dart';
2 import '../widgets/custom_card.dart';
3 import '../widgets/custom_textfield.dart';
4 class ReverseScreen extends StatefulWidget {
5   const ReverseScreen({super.key});
6   static const routeName = '/reverse';
7   @override
8   State<ReverseScreen> createState() => _ReverseScreenState();
9 }
```

Listing 9: lib/screens/reverse_screen.dart

```
1 import 'package:flutter/material.dart';
2 class TodoScreen extends StatefulWidget {
3   const TodoScreen({super.key});
4   static const routeName = '/todo';
5   @override
6   State<TodoScreen> createState() => _TodoScreenState();
7 }
```

Listing 10: lib/screens/todo_screen.dart

```
1 import 'package:flutter/material.dart';
2 import '../widgets/custom_card.dart';
3 import '../widgets/custom_textfield.dart';
4 import 'fake_call_screen.dart';
5 class CallScreen extends StatefulWidget {
6   const CallScreen({super.key});
7   static const routeName = '/call';
8   @override
9   State<CallScreen> createState() => _CallScreenState();
10 }
```

Listing 11: lib/screens/call_screen.dart

```
1 import 'package:flutter/material.dart';
2 class FakeCallScreen extends StatelessWidget {
3   final String name;
4   const FakeCallScreen({super.key, required this.name});
5   @override
6   Widget build(BuildContext context) => const SizedBox.shrink();
7 }
```

Listing 12: lib/screens/fake_call_screen.dart

```
1 import 'package:flutter/material.dart';
2 import '../widgets/custom_card.dart';
3 import '../widgets/custom_textfield.dart';
4 import 'fake_email_screen.dart';
5 class EmailScreen extends StatefulWidget {
6   const EmailScreen({super.key});
7   static const routeName = '/email';
8   @override
9   State<EmailScreen> createState() => _EmailScreenState();
10 }
```

Listing 13: lib/screens/email_screen.dart

```
1 import 'package:flutter/material.dart';
2 class FakeEmailScreen extends StatelessWidget {
3   final String sender;
4   const FakeEmailScreen({super.key, required this.sender});
5   @override
6   Widget build(BuildContext context) => const SizedBox.shrink();
7 }
```

Listing 14: lib/screens/fake_email_screen.dart

```
1 import 'dart:math';
2 import 'package:flutter/material.dart';
3 import '../widgets/custom_card.dart';
4 import '../widgets/custom_textfield.dart';
5 class QuizScreen extends StatefulWidget {
6   const QuizScreen({super.key});
7   static const routeName = '/quiz';
8   @override
9   State<QuizScreen> createState() => _QuizScreenState();
10 }
```

Listing 15: lib/screens/quiz_screen.dart


```
1 import 'package:flutter/material.dart';
2 class ZakahScreen extends StatefulWidget {
3   const ZakahScreen({super.key});
4   static const routeName = '/zakah';
5   @override
6   State<ZakahScreen> createState() => _ZakahScreenState();
7 }
```

Listing 16: lib/screens/zakah_screen.dart

Reusable Widgets

```
1 import 'package:flutter/material.dart';
2 class CustomTextField extends StatelessWidget {
3   const CustomTextField({
4     required this.controller,
5     required this.hintText,
6     required this.icon,
7     super.key,
8   });
9   final TextEditingController controller;
10  final String hintText;
11  final IconData icon;
12  @override
13  Widget build(BuildContext context) => const SizedBox.shrink();
14 }
```

Listing 17: lib/widgets/custom_textfield.dart

```
1 import 'package:flutter/material.dart';
2 class CustomCard extends StatelessWidget {
3   const CustomCard({required this.child, super.key});
4   final Widget child;
5   @override
6   Widget build(BuildContext context) => child;
7 }
```

Listing 18: lib/widgets/custom_card.dart